

# Predicting Market Outcomes from Earnings Call Transcripts: Fine-tuning, Retrieval, and Prompting

Surya Suresh

Nikhil Kumar

Dheeraj Gosula

## Abstract

We study whether a modern open-weight large language model can predict financial outcomes from S&P 500 earnings call transcripts, and which adaptation techniques actually help. We test the model on two binary classification tasks: stock direction (whether the price moves up or down over the three trading days after the call) and earnings surprise (whether reported EPS beats analyst consensus). For each task we evaluate three techniques: LoRA fine-tuning, retrieval-augmented generation (RAG), and few-shot prompting. Using Qwen2.5-7B-Instruct as a fixed base model, we run every combination of these techniques on both tasks and report results across 16 conditions. LoRA fine-tuning produces a substantial improvement on the surprise task (AUC 0.637 to 0.812, 95% CI excluding random) but no measurable improvement on stock direction (AUC remains near 0.50). Few-shot prompting and retrieval show smaller, mixed effects: they tend to help the unadapted base model but slightly hurt the fine-tuned one. The asymmetry between the two tasks suggests that LLMs fine-tuned on earnings calls can identify outcomes already discussed in the transcript, but that predicting stock movement from text alone remains close to chance.

## 1 Introduction

Earnings calls are quarterly events where a public company’s management reports financial results and answers questions from analysts. The transcripts contain financial figures, forward-looking guidance, and discussion of the quarter. Investors read them widely, but pulling predictive signal out of them is hard: stock returns over a few days are noisy, and useful content is mixed in with a lot of filler.

Most prior work on earnings call NLP relies on sentiment scoring or single-task classifiers (Loughran and McDonald, 2011; Araci, 2019;

Qin and Yang, 2019). More recent work has applied larger transformer architectures (Koval et al., 2023; Ma et al., 2020), but no study has systematically compared the three techniques most commonly used to adapt large language models to a certain domain: supervised fine-tuning, retrieval-augmented generation (RAG) (Lewis et al., 2020), and few-shot prompting (Brown et al., 2020; Wei et al., 2022).

We close this gap by running every combination of fine-tuning, RAG, and few-shot prompting on two binary classification tasks: stock direction (whether the price moves up or down over the three trading days following the call) and earnings surprise (whether reported EPS beats analyst consensus). We use Qwen2.5-7B-Instruct (Qwen Team, 2024) as a fixed base model and adapt it with QLoRA (Detrmers et al., 2023; Hu et al., 2022). All training and evaluation ran on a single 16 GB V100 GPU on OSC’s Pitzer cluster, the only cluster where we could reliably get GPU jobs scheduled. We report accuracy, balanced accuracy, and AUC with confidence intervals obtained by resampling each test set 1000 times, and describe a truncation strategy that fits long transcripts into the model’s context budget while keeping the segments most relevant to each task.

## 2 Related Work

**Sentiment-based approaches.** The earliest text-based methods for financial prediction use sentiment dictionaries such as Loughran-McDonald (Loughran and McDonald, 2011) or domain-specific classifiers like FinBERT (Araci, 2019; Yang et al., 2020). These models compress an entire transcript into a small number of sentiment scores, which are fed to a downstream classifier. The compression discards structural information, and as we confirm in Section 6, reduces stock direction prediction to

roughly chance accuracy.

**Earnings call deep learning.** [Qin and Yang \(2019\)](#) use multimodal text-and-audio features to predict post-call return volatility. [Ma et al. \(2020\)](#) combine GloVe embeddings with industry-sector signals to predict next-day stock movement, reporting 48.5–56.8% accuracy across sectors. [Koval et al. \(2023\)](#) apply hierarchical FinBERT to earnings surprise classification and report 76.56% accuracy on a balanced test set. [Larcker and Zakolyukina \(2012\)](#) study Q&A portions of conference calls and detect linguistic signals of executive deception that predict future financial restatements, showing that unscripted Q&A exchanges carry meaningful predictive content. Each of these works tests a single architecture on a single task. None of them compare across modern adaptation techniques.

**Domain LLMs and parameter-efficient adaptation.** BloombergGPT ([Wu et al., 2023](#)) pretrains a 50B-parameter model on Bloomberg’s proprietary financial corpus. We instead use a smaller, openly available model (Qwen2.5-7B) and adapt it with LoRA ([Hu et al., 2022](#)) and QLoRA ([Dettmers et al., 2023](#)), which combines 4-bit weight quantization with low-rank trainable adapters and fits fine-tuning on a single 16 GB GPU. Generic adaptation techniques like retrieval ([Lewis et al., 2020](#)) and chain-of-thought prompting ([Wei et al., 2022](#)) are well studied in open-domain NLP but have not been tested side-by-side on earnings call outcome prediction.

### 3 Data

We use the publicly available S&P 500 earnings transcripts corpus `Bose345/sp500_earnings_transcripts` on HuggingFace, which contains tens of thousands of calls. We fetch labels for each transcript via the `yfinance` Python library. For stock direction, we fetch the closing price on the last trading day before the call and on the third trading day after. The label is 1 (up) if the post-call price exceeds the pre-call price, 0 (down) otherwise. For earnings surprise, we use `yfinance`’s `get_earnings_dates()` method to retrieve reported EPS and analyst consensus, requiring a match within 10 days of the call. The label is 1 (beat) if reported  $\geq$  consensus, 0 (miss) otherwise.

We ran this pipeline twice. The first pass sampled the most recent 5,000 transcripts and recovered

| Set   | Task      | Total  | Train  | Test  | % pos. |
|-------|-----------|--------|--------|-------|--------|
| Small | Direction | 1,965  | 1,768  | 197   | 51.1   |
|       | Surprise  | 827    | 744    | 83    | 80.9   |
| Large | Direction | 11,449 | 10,304 | 1,145 | 53.0   |
|       | Surprise  | 7,838  | 7,054  | 784   | 79.3   |

Table 1: Dataset sizes for both pipeline runs after fetching labels. Train/test split is chronological 90/10 (no shuffle). % *pos.* is the fraction labeled *up* (direction) or *beat* (surprise).

only 1,965 valid direction labels and 827 valid surprise labels. The bottleneck was `yfinance`’s per-IP rate limit silently dropping requests. For the second pass we added a 1.5-second delay between calls and iterated newest-first, so any throttling drops older records rather than recent ones, and we sampled 12,000 transcripts for direction and 8,000 for surprise. This recovered 11,449 direction labels and 7,838 surprise labels. We refer to these as the *small* and *large* datasets, and report fine-tuning results on both.

The whole dataset is built once as a single batch job before any training begins. A build script samples transcripts from the corpus, fetches the labels described above, strips recurring boilerplate from each transcript (operator handoffs, “next question from” intros between Q&A speakers, closing remarks, and forward-looking disclaimers, which removes about 3.5% of characters without touching substantive content), and writes one JSONL record per labeled call with the fields the model consumes at training time. Training and evaluation read these JSONL files directly, so the expensive label-fetching and rate-limit handling are paid once up-front rather than repeated for every experiment.

Table 1 summarizes the two datasets. Direction is nearly class-balanced under both. Surprise is heavily skewed toward beat, reflecting the well-known tendency of S&P 500 firms to beat consensus more often than they miss. We mitigate this during training (Section 4.2) but preserve the natural distribution at test time so that reported metrics are not inflated by the class prior.

For each task we sort calls by date and use the earliest 90% for training and the latest 10% for testing. A random split would leak information from later calls into earlier predictions and inflate test performance, particularly for direction where the macro market regime itself carries real signal. Chronological splitting prevents this lookahead bias.

Transcripts are long. The median length under the Qwen tokenizer is approximately 13,000 tokens, far exceeding our 2,048-token budget per training example, and the boilerplate-stripped transcript is still typically far longer than the budget. Naive front-truncation (keep first 2,048 tokens) preserves the prepared opening remarks but discards the entire Q&A section. The Q&A is unscripted, contains analyst pressure on specific concerns, and is widely cited as a high-signal segment of the call (Qin and Yang, 2019; Larcker and Zakolyukina, 2012). We instead use a head-and-tail truncation: keep the first 60% (1,229 tokens) and the last 40% (819 tokens) of the budget, dropping only the middle. This preserves both the management narrative and the Q&A. The same truncation is used at training and at evaluation so the prompt distribution seen by the model is consistent across the two phases.

## 4 Methodology

### 4.1 Task formulation

Each prompt is constructed as

<system instruction>

Transcript:

<truncated transcript>

Answer:

where <system instruction> is a one-paragraph description of the task with the allowed answer tokens (*up/down* or *beat/miss*). The model is asked to emit one token after Answer:.

At inference we run a single forward pass and read the logits at the final position. We then take just the two logits corresponding to the answer tokens (for example, *up* and *down*) and apply softmax across that pair to get a probability for the positive class. We never generate text and never parse free-form output. The same scoring procedure is used for the base model and every augmented variant, so all conditions are directly comparable.

### 4.2 Fine-tuning with QLoRA

We fine-tune Qwen2.5-7B-Instruct (Qwen Team, 2024) using QLoRA (Dettmers et al., 2023), which combines 4-bit weight quantization with low-rank adapter training (Hu et al., 2022). The base model is loaded with 4-bit NF4 quantization and double quantization, reducing weight memory from about 14 GB (fp16) to about 4 GB. This is small enough to fit the 7B model on a 16 GB V100 along

with activations and gradients. We attach LoRA adapters to the four attention projection matrices in every transformer block (q\_proj, k\_proj, v\_proj, o\_proj) at rank  $r = 16$ , scaling  $\alpha = 32$ , and dropout 0.05. Feed-forward and embedding layers are frozen, leaving approximately 10 M trainable parameters out of 7.6 B total (0.13%). We adapt only the attention projections because attention is what controls how tokens combine information across positions, which is where domain adaptation needs to take effect (for example, attending more strongly to Q&A guidance language).

We train each task’s adapter independently for 3 epochs with the AdamW optimizer in 8-bit paged form. Learning rate is  $2e-4$  with cosine decay and 3% linear warmup. The effective batch size is 8, processed one example at a time with gradients accumulated over 8 steps before each weight update so we can fit within V100 memory. Mixed precision is fp16. We use gradient checkpointing to keep memory usage in check on the V100, and the training loss is computed only on the answer token rather than on the whole prompt. For surprise (roughly 80% beat, 20% miss), without intervention the model just learns to predict the majority class on every example. To prevent this we balance the training split after the train/test split by sampling miss examples with replacement until they match the count of beat examples. We do this only on the training split. The validation and test splits keep their natural imbalance. We evaluate on the chronological validation set after each epoch and retain the checkpoint with the lowest validation loss, which protects against overfitting on the smaller surprise dataset.

Each fine-tuned model is saved as a  $\sim 40$  MB LoRA adapter alongside the tokenizer. The base model weights themselves are never modified, so the adapter is a small standalone file that we attach back on top of the original Qwen2.5-7B at inference. We load the base with the same 4-bit quantization and attach the adapter via the PEFT library. The same procedure is used for the RAG and few-shot variants that build on top of the fine-tuned model.

### 4.3 Few-shot prompting

[Section to be completed by teammate.] The few-shot variant prepends  $k$  labeled examples drawn chronologically from the training split before the test transcript. Per the team’s design,  $k = 4$  shots are used, balanced across labels (2 positive, 2 nega-

tive), with 150 tokens per shot and a 3072-token total prompt budget. Chain-of-thought reasoning was tested but did not yield reliable improvements in our setting and is excluded from the final pipeline, likely because the additional reasoning step disturbs the single-token answer format the adapter was trained on. Extractive sentence selection using a 24-keyword financial dictionary is used to densify each shot’s transcript content. Final scoring uses the same softmax-over-target-pair procedure described in Section 4.1.

#### 4.4 Retrieval-augmented generation

[Section to be completed by teammate.] The retrieval corpus contains  $\sim 450$  documents, one per training transcript, each consisting of the company ticker, call date, and first 2,000 characters of the transcript. Retrieval is sparse: the system first attempts a same-company prior-quarter lookup (most recent transcripts from the same ticker with date strictly preceding the query call), then falls back to BM25 keyword ranking over the full training corpus to fill remaining slots. Retrieved passages are prepended to the prompt within a  $\sim 400$ -token budget. The corpus is restricted to the training split to prevent any test leakage.

### 5 Experimental Setup

All training and evaluation runs use a single NVIDIA V100 PCIE 16 GB GPU on the OSC Pitzer cluster, with the Python environment pinned to PyTorch 2.3.1+cu118 (the latest version that still supports the V100), CUDA 11.8, transformers 4.44.2, peft 0.13.2, and bitsandbytes 0.43.3.

For each task we evaluate eight conditions formed by toggling fine-tuning, RAG, and few-shot prompting: zero-shot base, fine-tuned only, RAG only, prompting only, and the four pairwise and triple combinations. The base-only, RAG-only, and prompting-only variants all share the same set of base-model weights. The fine-tuned variants use a task-specific adapter on top of that base.

For each condition we report three metrics. Plain accuracy (prediction-vs-label match at threshold 0.5) is reported but not relied on for claims, because when the test set is imbalanced, accuracy can be high just from guessing the majority class on every example. Balanced accuracy is the mean of per-class recall,  $(\text{recall}_+ + \text{recall}_-)/2$ , which is robust to class imbalance. AUC-ROC with a 95% bootstrap confidence interval (1,000 resamples) is

our primary metric. It is threshold-free, robust to imbalance, and the CI tells us whether the gap from random (0.50) is real or could just be noise from a small test set. We call a result above chance when its 95% AUC CI excludes 0.50, and we call one model better than another when their CIs do not overlap. If the CIs overlap, we report the gap but note that it could be noise.

## 6 Results

### 6.1 Fine-tuning vs. zero-shot base

Table 2 presents fine-tuning results on both the small and large datasets.

| Set   | Task      | Model | Acc          | B-Acc        | AUC          | 95% CI              |
|-------|-----------|-------|--------------|--------------|--------------|---------------------|
| Small | Direction | Base  | 0.569        | 0.502        | 0.493        | [0.41, 0.57]        |
|       | Direction | FT    | 0.594        | 0.500        | 0.499        | [0.42, 0.58]        |
|       | Surprise  | Base  | 0.759        | 0.566        | 0.637        | [0.49, 0.78]        |
|       | Surprise  | FT    | <b>0.807</b> | <b>0.708</b> | <b>0.812</b> | <b>[0.69, 0.92]</b> |
| Large | Direction | Base  | 0.542        | 0.503        | 0.500        | [0.47, 0.53]        |
|       | Direction | FT    | 0.457        | 0.492        | 0.499        | [0.46, 0.53]        |
|       | Surprise  | Base  | 0.739        | 0.575        | 0.620        | [0.57, 0.66]        |
|       | Surprise  | FT    | <b>0.755</b> | <b>0.641</b> | <b>0.737</b> | <b>[0.70, 0.78]</b> |

Table 2: Fine-tuning vs. zero-shot base on the small and large datasets. *B-Acc* is balanced accuracy. AUC CIs are bootstrap with 1,000 resamples. Test sizes: small direction  $n = 197$ , small surprise  $n = 83$ , large direction  $n = 1,145$ , large surprise  $n = 784$ .

On the surprise task, fine-tuning improves AUC on both datasets. On the small dataset the gain is 0.18 ( $0.637 \rightarrow 0.812$ ) and balanced accuracy improves by 0.14 ( $0.566 \rightarrow 0.708$ ). The fine-tuned AUC’s 95% CI  $[0.69, 0.92]$  sits entirely above random chance (0.50), but the wide interval reflects the small test size ( $n = 83$ ). On the large dataset the gain is 0.12 ( $0.620 \rightarrow 0.737$ ) with much tighter CIs from the larger test set ( $n = 784$ ). The fine-tuned CI  $[0.70, 0.78]$  does not overlap with the base CI  $[0.57, 0.66]$ , so the improvement is unlikely to be noise. The smaller gain on the larger test set is likely closer to the true effect, with the small-dataset 0.812 inflated by sampling noise.

On the direction task, both base and fine-tuned models hover near AUC 0.50 on both datasets with CIs that include 0.50. On the small dataset the fine-tuned model’s plain accuracy (0.594) matches the majority-class baseline ( $117/197 \approx 0.594$ ) almost exactly, indicating the model collapsed onto predicting “up” on every example. Balanced accuracy is exactly 0.500, confirming this collapse. On the large dataset the fine-tuned accuracy (0.457) ac-



tually drops below the majority baseline (0.551), but AUC (0.499) and balanced accuracy (0.492) confirm the same chance-level result. The larger test set tightens the AUC CI to about  $\pm 0.03$ , giving us high confidence that the fine-tuned model is not above chance on direction. Despite minority-class oversampling during training, the model could not extract any directional signal beyond the class prior on either dataset.

These results line up with prior work. [Ma et al. \(2020\)](#) report 48.5–56.8% accuracy across sectors for next-day stock movement using GloVe and industry features, similar to our chance-level direction result despite their use of explicit sector signals. [Koval et al. \(2023\)](#) report 76.56% accuracy on a balanced earnings surprise test set with hierarchical FinBERT. Our 75.5% accuracy is not the right comparison because our test set is heavily skewed (76.7% beat). A model that just predicts “beat” on every example would already score around 77% without learning anything, so the raw number does not tell us much on its own. Our balanced accuracy is 0.641 and our AUC is 0.737. Both numbers are below Koval’s 0.7656 but show that the model has learned real signal on this task and lands in the same range as their hierarchical FinBERT. We do not claim a head-to-head comparison since the test sets and label-fetching protocols differ.

## 6.2 Few-shot prompting

*[Analysis to be completed by teammate.]* The 4-shot prompting results are presented in Table 3 in the Discussion alongside the other augmentation conditions.

## 6.3 Retrieval-augmented generation

*[Analysis to be completed by teammate.]* The combined results across all conditions, including RAG, are presented in Table 3.

| Set   | Task      | Model | ZS    | 4-shot | RAG   | RAG+4 |
|-------|-----------|-------|-------|--------|-------|-------|
| Small | Direction | Base  | 0.493 | 0.520  | 0.492 | 0.532 |
|       | Direction | FT    | 0.499 | 0.536  | 0.494 | 0.538 |
|       | Surprise  | Base  | 0.637 | 0.720  | 0.680 | 0.676 |
|       | Surprise  | FT    | 0.812 | 0.748  | 0.752 | 0.722 |
| Large | Direction | Base  | 0.500 | –      | –     | –     |
|       | Direction | FT    | 0.499 | –      | –     | –     |
|       | Surprise  | Base  | 0.620 | –      | –     | –     |
|       | Surprise  | FT    | 0.737 | –      | –     | –     |

Table 3: AUC across all conditions on the small and large datasets. ZS = zero-shot, 4-shot = 4-shot prompting, RAG = retrieval, RAG+4 = both.

# 7 Discussion

## 7.1 Why fine-tuning helps surprise but not direction

Surprise is reflected in how management discusses the quarter. The model has to parse dense financial text, work through forward guidance, and read the pushback analysts apply in Q&A. Even with all of this content available in the transcript, the base model only reaches AUC 0.637, which shows the relevant signal is not sitting on the surface for a general-purpose LLM. Fine-tuning closes the gap by teaching the model which language patterns line up with a beat versus a miss across thousands of calls, and that is where the improvement comes from. The task is a real test of financial language understanding rather than a memorization task.

Direction depends on what investors do over the three trading days after the call. That comes from broader market activity and news outside the call itself. None of that is in the transcript. The model is being asked to forecast something whose driver lives outside the input, and our results show that text-only models fail at it. Prior work on post-earnings announcement drift reports only small gains over chance on short-window direction even with audio and structured market data ([Qin and Yang, 2019](#); [Sawhney et al., 2020](#)), and our text-only setup has even less information to work with.

## 7.2 RAG and prompting on the small dataset

Table 3 shows a clear pattern on the small dataset. Without fine-tuning, augmentations help on surprise: AUC goes from 0.637 (zero-shot) to 0.720 with 4-shot prompting and 0.680 with RAG. With fine-tuning, the same augmentations slightly hurt: 0.812 drops to 0.748, 0.752, and 0.722 with 4-shot, RAG, and both. The fine-tuned adapter has already learned what to look for in the transcript itself, so

adding extra context just crowds out the most informative parts of the test transcript within a fixed context window. For direction, no condition rises meaningfully above chance, which says the bottleneck is signal in the input, not modeling choices.

### 7.3 Class imbalance and metric choice

The surprise task is roughly 77% beat. A model that always predicts “beat” gets about 77% accuracy without learning anything, close to the base model’s 75.9% on the small dataset and 73.9% on the large. We rely on AUC and balanced accuracy because they cannot be inflated by always picking the majority class. Direction shows the same point in reverse: the small fine-tuned accuracy of 0.594 looks like a gain over base 0.569, but it exactly matches the majority baseline of 117/197, and balanced accuracy of 0.500 confirms the model is predicting “up” on every example.

## 8 Limitations

**Temporal coverage.** The small dataset spans about one year of recent calls (May 2024–May 2025). The large dataset reaches further back: direction covers October 2019–May 2025 (about 5.5 years across multiple market conditions), and surprise covers July 2021–May 2025 (about 3.8 years). Performance on calls outside this window or under future macro conditions is unknown.

**Single base model.** We did not run an ablation across base LLMs. Llama-3-8B, Gemma-2-9B, and finance-pretrained models like FinMA may behave differently. Whether the answer is in the transcript does not depend on which LLM we use, so the high-level conclusion that fine-tuning helps surprise but not direction would likely hold. The specific AUC numbers, however, would shift.

**Truncation budget.** Our 2,048-token cap is what fit on the V100 GPU on Pitzer with our QLoRA configuration. Attempting 4,096 tokens caused out-of-memory errors. Extending context to 4,096 or 8,192 tokens would let us include more of the middle of each call and is a natural follow-up on a larger GPU.

**Augmentations only on the small dataset.** The 4-shot prompting and RAG conditions in Table 3 were run only on the small dataset. Large-dataset versions are not included here, so the augmentation analysis in the Discussion should be read as preliminary for the larger setting.

**Not a forecasting task.** Surprise prediction here is a language-understanding test rather than a forecasting one. The model classifies completed calls based on their content, so the result is evidence the model can read financial language well, not that it would generate signals tradeable in real time.

## 9 Conclusion

We tested whether a modern open-weight LLM, adapted via fine-tuning, retrieval, or prompting, can predict financial outcomes from earnings call transcripts. We ran the experiments on two datasets: a small one with about 2,000 labeled calls and a larger one with about 11,000 direction and 8,000 surprise labels. LoRA fine-tuning improves EPS surprise prediction on both (AUC 0.812 vs. 0.637 base on the small set, 0.737 vs. 0.620 on the large set with much tighter confidence intervals). Stock direction prediction stays at chance on both datasets. Surprise is reflected in how management discusses the quarter, so the model can pick it up from text alone. Three-day stock direction depends on what investors do after the call, which is outside the transcript. On the small dataset we also tested 4-shot prompting and RAG. Augmentations help the base model but slightly hurt the fine-tuned one, which suggests the adapter has already learned what to attend to in the transcript. Our results support the use of fine-tuned LLMs for transcript-grounded financial classification, and caution against expecting them to forecast short-window market reactions from text alone.

## Author Contributions

**Surya Suresh.**

**Nikhil Kumar.** Built the large dataset, including the rate-limit-aware label fetching and the boilerplate stripping. Designed the head-and-tail truncation method. Implemented and ran all QLoRA fine-tuning and evaluation on both the small and large datasets. Contributed to the writing of the report, with primary responsibility for the fine-tuning section.

**Dheeraj Gosula.**

## Code

The full implementation for all parts of this project, including the data pipeline, fine-tuning, retrieval, prompting, and evaluation code, is avail-

able at <https://github.com/surya-suresh/financialnlp>.

## Acknowledgements

AI assistants (Claude and ChatGPT) were used to help organize and format the LaTeX code for this report, and as a mild writing assistant.

## References

- Dogu Araci. 2019. FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. *arXiv preprint arXiv:1908.10063*.
- Tom Brown, Benjamin Mann, Nick Ryder, et al. 2020. Language Models are Few-Shot Learners. In *NeurIPS*.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *NeurIPS*.
- Edward Hu, Yelong Shen, Phillip Wallis, et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*.
- Ross Koval, Nicholas Andrews, and Xifeng Yan. 2023. Forecasting Earnings Surprises from Conference Call Transcripts. In *Findings of ACL*.
- David F. Larcker and Anastasia A. Zakolyukina. 2012. Detecting Deceptive Discussions in Conference Calls. *Journal of Accounting Research*, 50(2):495–540.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*.
- Tim Loughran and Bill McDonald. 2011. When Is a Liability Not a Liability? Textual Analysis, Dictionaries, and 10-Ks. *Journal of Finance*, 66(1):35–65.
- Yu Ma, Rui Mao, Qika Lin, et al. 2020. Multi-Source Aggregated Classification for Stock Price Movement Prediction. *Information Fusion*.
- Yao Qin and Yi Yang. 2019. What You Say and How You Say It Matters: Predicting Stock Volatility Using Verbal and Vocal Cues. In *ACL*, pages 390–401.
- Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.
- Ramit Sawhney, Piyush Khanna, Arshiya Aggarwal, Taru Jain, Puneet Mathur, and Rajiv Ratn Shah. 2020. VolTAGE: Volatility Forecasting via Text Audio Fusion with Graph Convolution Networks for Earnings Calls. In *EMNLP*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*.
- Shijie Wu, Ozan Irsoy, Steven Lu, et al. 2023. BloombergGPT: A Large Language Model for Finance. *arXiv preprint arXiv:2303.17564*.
- Yi Yang, Mark Christopher Siy Uy, and Allen Huang. 2020. FinBERT: A Pretrained Language Model for Financial Communications. *arXiv preprint arXiv:2006.08097*.